

Universidade Federal de Goiás

Instituto de Matemática e Estatística e Instituto de Informática
(INF/UFG)

Curso: Matemática Aplicada e Computacional

Disciplina: INF0286 – Algoritmos e Estruturas de Dados I

Professora: Dra. Renata Dutra Braga



Noções de Complexidade de Algoritmos

Por que alguns programas são rápidos e outros lentos?

Objetivo: Entender como medir a eficiência de um algoritmo em termos de tempo e espaço.



Por que estudar complexidade?

- ❑ Nem todos os algoritmos resolvem um mesmo problema com a mesma eficiência
- ❑ Computadores têm limites físicos: tempo de CPU, memória, energia
- ❑ Precisamos saber qual algoritmo é melhor antes de implementá-lo

❑ Exemplo: Procurar um nome em uma lista de 1.000.000 de pessoas:

- **Busca linear:** comparar um por um.
- **Busca binária:** cortar a lista ao meio a cada passo.

O que é um algoritmo?

Definição: Conjunto finito de instruções bem definidas para resolver um problema.

Exemplo: calcular o fatorial de n.

```
int fatorial(int n) {  
    int r = 1;  
    for (int i = 1; i <= n; i++)  
        r *= i;  
    return r;  
}
```

Pergunta: O que acontece se o n dobrar?

O tempo também dobra → Complexidade $O(n)$.

O que é Complexidade de Algoritmo

Mede o custo computacional de um algoritmo conforme o tamanho da entrada.

Dois tipos principais:

- **Complexidade de tempo** – número de operações.
- **Complexidade de espaço** – memória usada.

Ponto-chave:

Não queremos medir o tempo real do relógio, mas sim a **taxa de crescimento do número de operações**.

Análise Assintótica

01

Estuda o comportamento do algoritmo quando $n \rightarrow \infty$

02

Ignora detalhes de hardware e constantes

03

Foco no crescimento

Exemplo:

```
for (int i = 0; i < n; i++)  
    printf("Oi");
```

Executa n vezes $\rightarrow O(n)$. Se tivesse $2n + 5$, ainda seria $O(n)$ (constantes são desprezadas).

O que significa dizer que "constantes são desprezadas"?

Quando analisamos a **complexidade de tempo**, queremos entender **como o tempo cresce quando o tamanho da entrada (n) aumenta muito**. Não estamos interessados no tempo exato (em segundos), mas no **comportamento do crescimento**.

Matematicamente, a função que descreve o número de operações é:

$$f(n) = 2n + 5$$

Mas quando n cresce muito (digamos, 1.000.000 ou 1 bilhão):

- o termo **$2n$** domina completamente o comportamento da função;
- o **$+5$** é insignificante em comparação a $2n$;
- e o **2** é apenas um fator constante.

Assim, para *grandes* n , dizer que algo custa " $2n + 5$ operações" é praticamente o mesmo que dizer que custa " n operações".

Por isso dizemos que **$O(2n + 5) = O(n)$** .

Tipos de Complexidade de Tempo

Tipo	Nome	Descrição	Exemplo
$O(1)$	Constante	Independente do tamanho da entrada	Acessar vetor[0]
$O(\log n)$	Logarítmica	Reduz o problema pela metade	Busca binária
$O(n)$	Linear	Cresce proporcionalmente	Percorrer vetor
$O(n \log n)$	Quase linear	Combina divisão e fusão	Merge Sort
$O(n^2)$	Quadrática	Duas iterações aninhadas	Bubble Sort
$O(2^n)$	Exponencial	Crescimento explosivo	Força bruta (subconjuntos)

Exemplo $O(1)$: Constante

```
int primeiroElemento(int v[]) {  
    return v[0];  
}
```

Sempre realiza a mesma quantidade de operações, independentemente de n .

Complexidade: $O(1)$

Analogia: abrir o primeiro livro de uma pilha, não importa o tamanho dela.

Exemplo $O(n)$: Linear

```
int soma(int v[], int n) {  
    int s = 0;  
    for (int i = 0; i < n; i++)  
        s += v[i];  
    return s;  
}
```

O número de operações cresce proporcionalmente ao tamanho do vetor.

Complexidade: $O(n)$

Analogia: contar um por um os livros de uma prateleira.

Exemplo $O(n^2)$: Quadrática

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++)  
        printf("%d ", i + j);
```

Cada elemento é comparado com todos os outros.

Complexidade: $O(n^2)$

Analogia: todos contra todos (torneio de xadrez com n jogadores).

Exemplo $O(\log n)$: Logarítmica

```
int buscaBinaria(int v[], int n, int x) {  
    int ini = 0, fim = n - 1;  
    while (ini <= fim) {  
        int meio = (ini + fim) / 2;  
        if (v[meio] == x) return meio;  
        if (v[meio] < x) ini = meio + 1;  
        else fim = meio - 1;  
    }  
    return -1;  
}
```

A cada passo, o problema é dividido por 2.

Complexidade: $O(\log n)$

Analogia: procurar um número em uma lista telefônica.

Exemplo $O(n \log n)$: Quase linear

Merge Sort (simplificado):

```
void mergeSort(int v[], int n) {  
    // Divide o vetor em metades e ordena cada uma  
    // Depois junta (merge)  
}
```

Divide ($\log n$ vezes) e percorre (n vezes).

Complexidade: $O(n \log n)$

Analogia: dividir para conquistar.

Exemplo $O(2^n)$: Exponencial

```
int fibonacci(int n) {  
    if (n <= 1) return n;  
    return fibonacci(n-1) + fibonacci(n-2);  
}
```

Chamadas recursivas explodem.

Complexidade: $O(2^n)$

Analogia: dobrar tarefas a cada etapa — rapidamente impossível.

Comparação Gráfica

n	$O(\log n)$	$O(n)$	$O(n^2)$	$O(2^n)$
10	3	10	100	1024
20	4	20	400	1.048.576
50	6	50	2500	1.1259e15

Complexidade de Espaço

Mede memória utilizada em função do tamanho da entrada.

Exemplo:

```
int v[1000]; // O(1)
int v2[n];   // O(n)
int m[n][n]; // O(n2)
```

Importante: um algoritmo pode ser rápido, mas usar muita memória — o equilíbrio é essencial.

Casos da Análise de Tempo

Tipo	Descrição	Exemplo
Caso melhor (Best Case)	Situação mais rápida possível	Buscar o 1º elemento
Caso médio (Average Case)	Tempo esperado em geral	Elemento no meio
Caso pior (Worst Case)	Situação mais lenta	Elemento não está na lista

Normalmente usamos o **pior caso (O)** para garantir desempenho mínimo.

Notações Big-O, Ω e Θ

Notação	Nome	Indica	Tipo de Caso	Como Ler	Exemplo de leitura	Interpretação
O(g(n))	Big-O	Limite superior	Pior caso	“O de n²”	“O algoritmo é O de n ao quadrado”	Tempo no máximo igual a g(n)
Ω(g(n))	Ômega	Limite inferior	Melhor caso	“Ômega de n”	“O algoritmo é ômega de n”	Tempo no mínimo igual a g(n)
Θ(g(n))	Teta	Limite exato	Caso médio ou exato	“Teta de n log n”	“O algoritmo é teta de n log n”	Tempo cresce proporcional a g(n)

Exemplo: Se $T(n) = 3n^2 + 2n + 1$, então $\rightarrow O(n^2), \Omega(n^2), \Theta(n^2)$.

Big-O (O maiúsculo) – limite superior

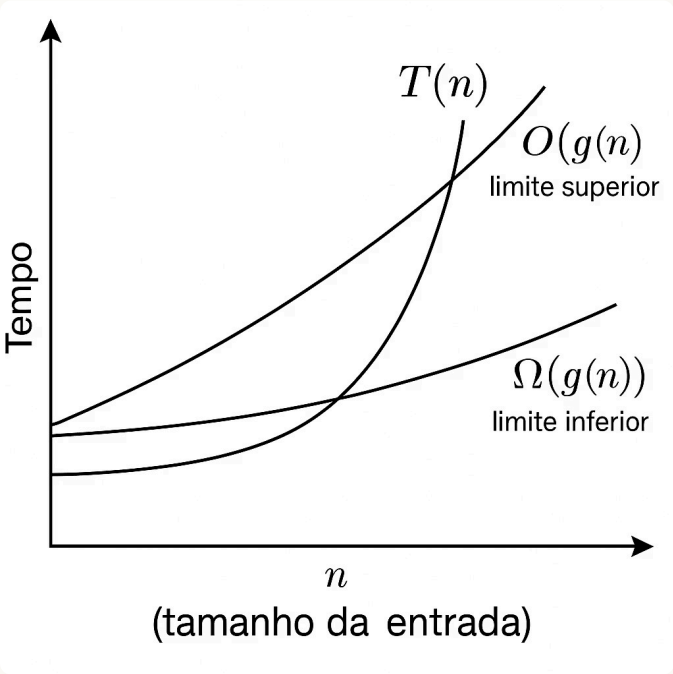
- Leitura: “O de n ao quadrado”, “O de n”, “O de log n”, etc. Símbolo: $O(g(n))$ Lê-se: “Ordem de g de n” ou “é O de g de n”.
- Significado: A notação Big-O fornece um limite superior para o crescimento do tempo de execução. Ela representa o pior caso: o algoritmo não cresce mais rápido que $g(n)$ (até uma constante multiplicativa).
- Exemplo: O algoritmo tem complexidade $O(n^2)$. Significa que o tempo de execução cresce no máximo proporcional a n^2 .
- Interpretação prática: Se um algoritmo é $O(n^2)$, ele pode ser mais rápido em alguns casos, mas nunca mais lento do que algo proporcional a n^2 , quando n é grande.

Ômega (Ω) – limite inferior

- Leitura: “Ômega de n”, “Ômega de log n”, etc. Símbolo: $\Omega(g(n))$ Lê-se: “Ômega de g de n”.
- Significado: A notação $\Omega(g(n))$ representa o limite inferior: indica o melhor caso, ou seja, o tempo de execução pelo menos cresce como $g(n)$.
- Exemplo: O algoritmo é $\Omega(n)$. Significa que o tempo de execução no mínimo cresce proporcionalmente a n .

Teta (Θ) – limite exato

- Leitura: “Teta de n”, “Teta de n^2 ”, etc. Símbolo: $\Theta(g(n))$ Lê-se: “Teta de g de n”.
- Significado: A notação $\Theta(g(n))$ indica o crescimento exato do algoritmo: ele cresce nem mais rápido, nem mais lento do que $g(n)$. É o caso em que o limite superior e o limite inferior coincidem.
- Exemplo: O algoritmo é $\Theta(n^2)$. Significa que o tempo de execução cresce proporcionalmente a n^2 , tanto no melhor quanto no pior caso.



Como analisar um algoritmo

01

Identifique as instruções executadas repetidamente

03

Ignore constantes e termos de menor ordem

Exemplo:

```
for (int i = 0; i < n; i++) // n vezes
    for (int j = 0; j < n; j++) // n vezes
        printf("%d", i+j);
```

$O(n \times n) = O(n^2)$.

02

Conte o número de repetições em função de n

04

Classifique a complexidade ($O(1)$, $O(n)$, etc.)

Atividade de Raciocínio

Compare os algoritmos abaixo:

// A

```
for (int i = 0; i < n; i++)  
    for (int j = i; j < n; j++)  
        printf("%d", i+j);
```

// B

```
for (int i = 0; i < n; i++)  
    printf("%d", i);
```

Qual tem menor complexidade? Discuta em grupo antes de responder.

Dicas de Estudo

Compare algoritmos resolvendo o mesmo problema

Sempre pense em como o tempo cresce com n

Visualize as curvas de crescimento

Teste com vetores grandes ($n = 1000, 10000\dots$) e observe o tempo

Conclusões

- Complexidade mede eficiência de tempo e espaço
 - A notação Big-O descreve o crescimento assintótico
 - Saber analisar algoritmos ajuda a escrever códigos mais rápidos e elegantes
 - Nem sempre o algoritmo mais simples é o mais eficiente!
-

Referências

- Feofiloff, P. Algoritmos em Linguagem C. Elsevier, 2009.
- Ziviani, N. Projeto de Algoritmos com Implementação em C. Thomson, 2010.
- Cormen, T. et al. Algoritmos: Teoria e Prática. Campus, 2002.
- Celes, W., Cerqueira, R., Rangel, J. Introdução a Estruturas de Dados. Elsevier, 2016.
- Backes, A. Algoritmos e Estruturas de Dados em C. LTC, 2023.